You Probably Don't Need (All of) OpenTelemetry

Colin Douch, SRE Lead

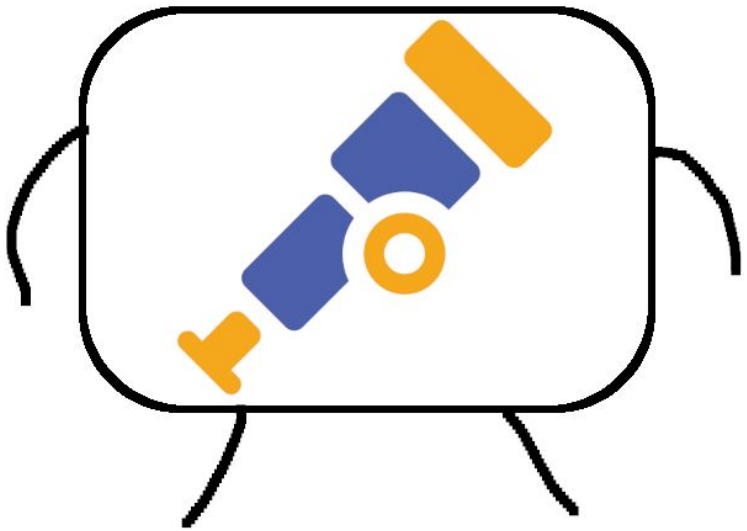
You Probably Don't Need (All of) OpenTelemetry

Colin Douch, SRE Lead

Auto-Instrument Everything you Need!

Switch Observability Vendors Easily!

Convenient!

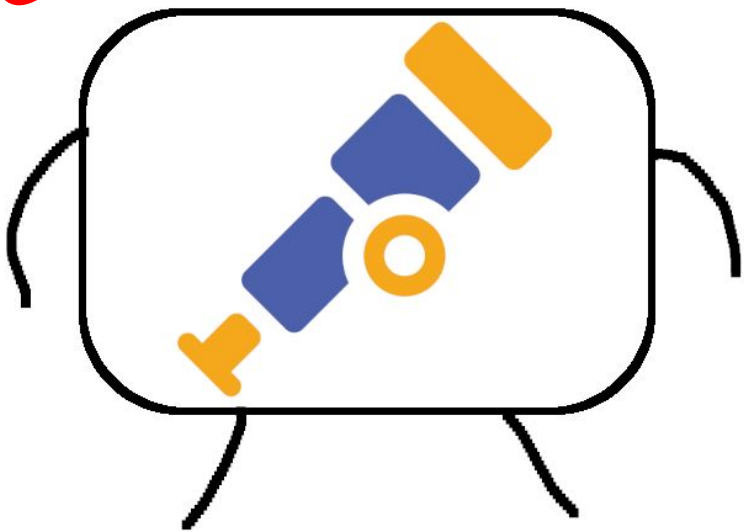


~~Auto-Instrument Everything you Need!~~

~~Convenient!~~

UNTRUTHS

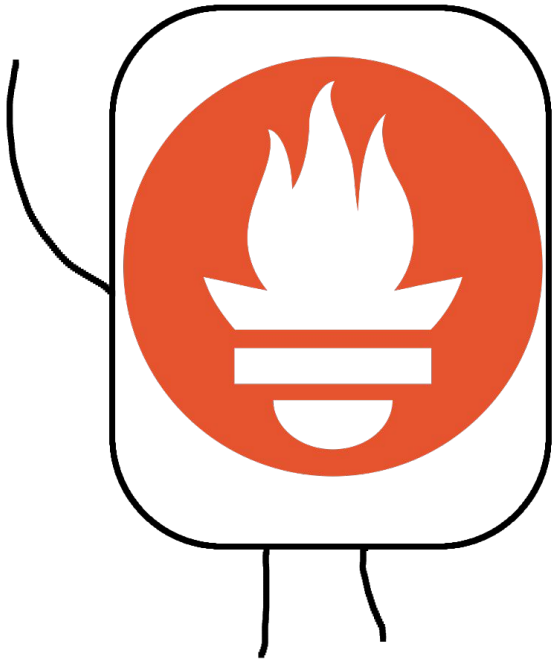
~~Switch Observability Vendors Easily!~~



Hi! I’m Colin

- Formerly formerly founded and led the Observability Team @ Cloudflare
- Formerly an SRE @ DuckDuckGo
- Currently leading an SRE team at a Fintech in London (and hiring! Come chat!)
- On the wrong side of the world





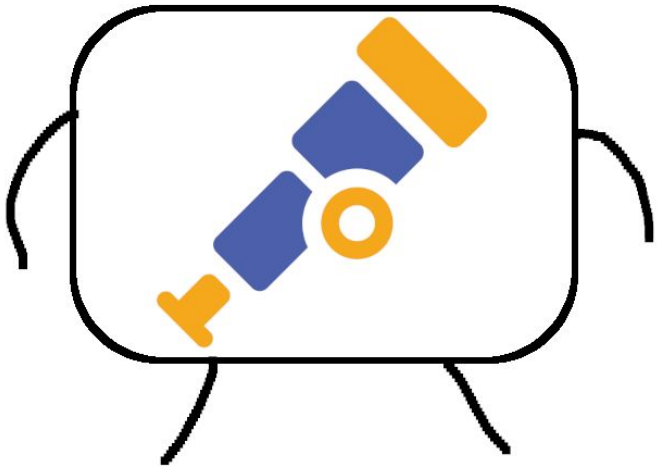
```
$ curl http://localhost:9090/metrics

# HELP process_cpu_seconds_total
Total user and system CPU time spent
in seconds.
# TYPE process_cpu_seconds_total
counterprocess_cpu_seconds_total
12.847
```

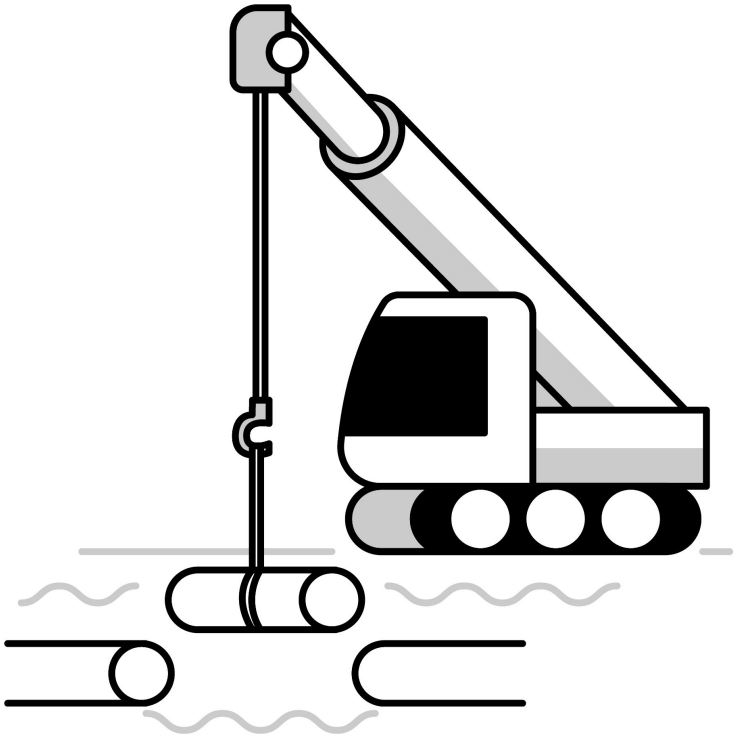
In Otel, if your telemetry isn’t flowing, you can…

- Add a debug exporter?
- Point to OTel CLI?
- Hope that your vendor can help you?

All of which require reinstrumentation…



Push based Protobuf wins us **parsing speed**, at the cost of ***debuggability***



Instrumentation?

For Prometheus:

- Format a string
- Serve it on a socket
- Maybe use a client library, if you want

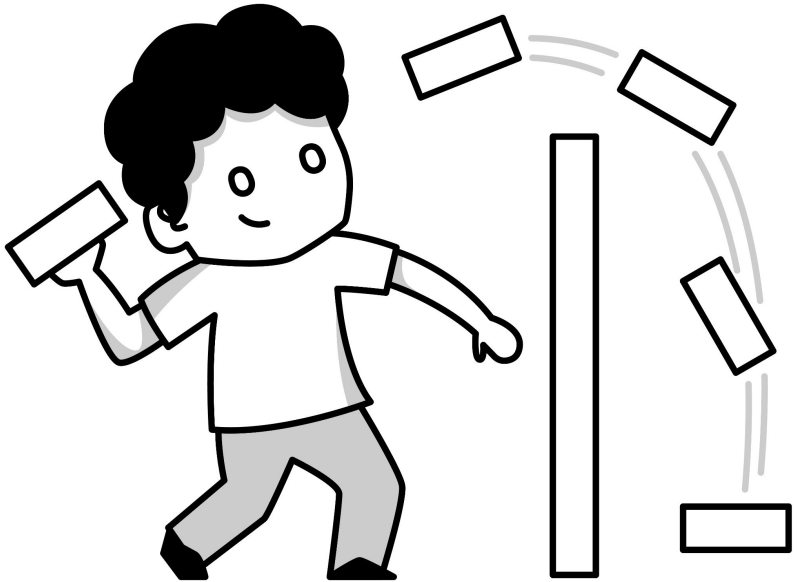
For OTel:

- An SDK
- A protobuf dependency
- An exporter to tune
- Maybe a collector, a batch processor, ratelimiting, transformations

StatsD was even simpler, but came with different semantics

```
echo "myapp.requests:1|c" | nc -u -w0
localhost 8125
```

OTel refuses to *choose*



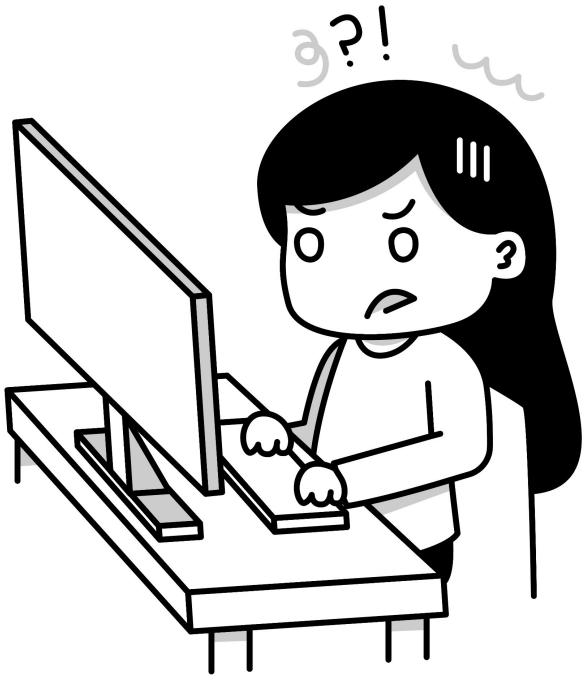
Statsd: aggregates at the sink

Prometheus: Expects the application to aggregate

Different usecases, and OTel hedges to support bothwith “temporalities”

And that’s sort of the theme

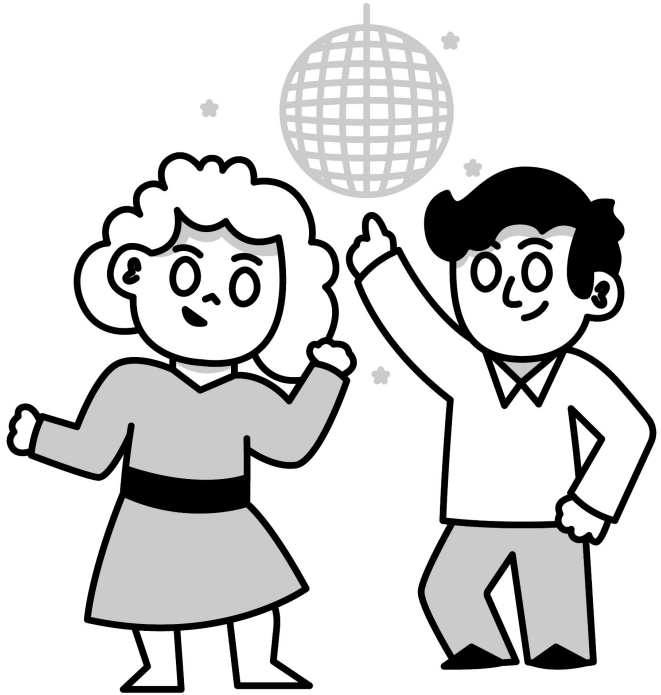

```
{
  "resourceMetrics": [{
    "scopeMetrics": [{
      "metrics": [{
        "name": "requests",
        "gauge": {
          "dataPoints": [{
            "asInt": "1",
            "timeUnixNano": "1758326400000
          }]
        }
      }]
    }]
  }]
}
```

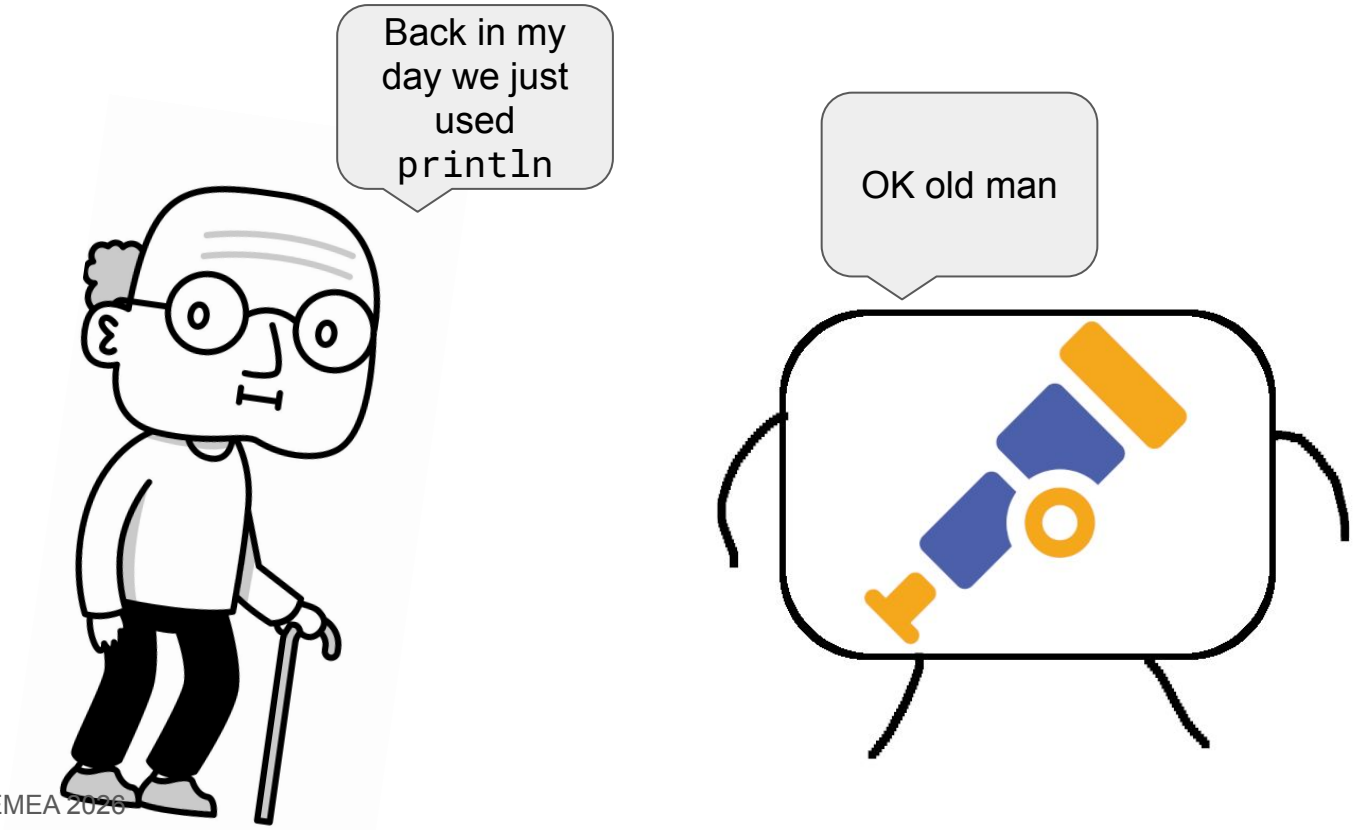



The slow part is transforming from OTLP to the internal representation, not in parsing

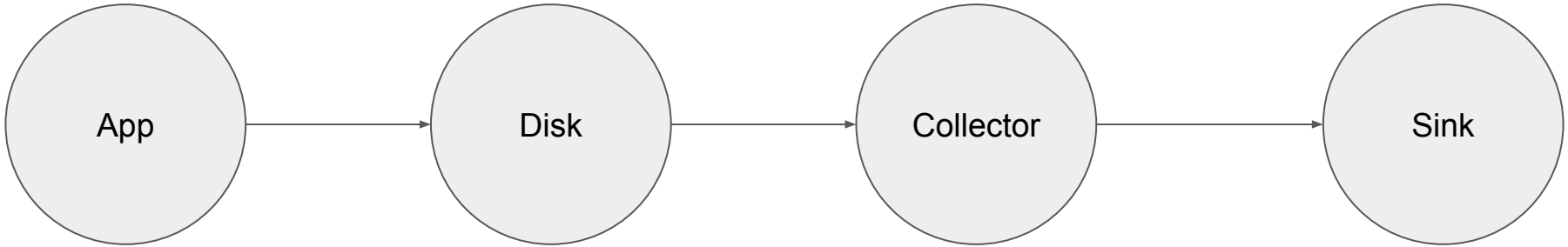


Let’s talk about logs




Back in my day we just used println
OK old man

This is a 12 factor app!



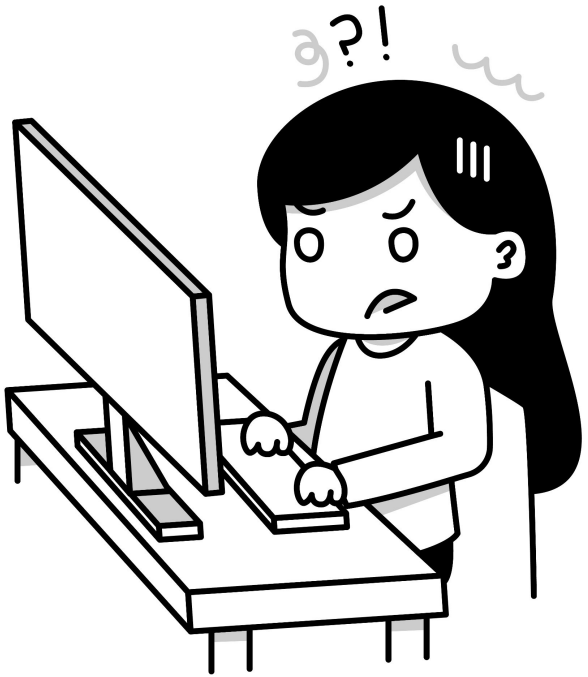
And again, we get schema inflation

```
{
   "ts":"2025-09-20T00:00:00Z",
   "level":"info",
   "service":"auth",
   "msg":"user logged in"
}
```

```
{
  "resourceLogs": [{
    "resource": {
      "attributes": [{
        "key": "service.name",
        "value": { "stringValue": "auth" }
      }]
    },
    "scopeLogs": [{
      "logRecords": [{
        "timeUnixNano": "1758326400000000000",
        "severityNumber": 9,
        "body": { "stringValue": "user logged in" }
      }]
    }]
  }]
}
```

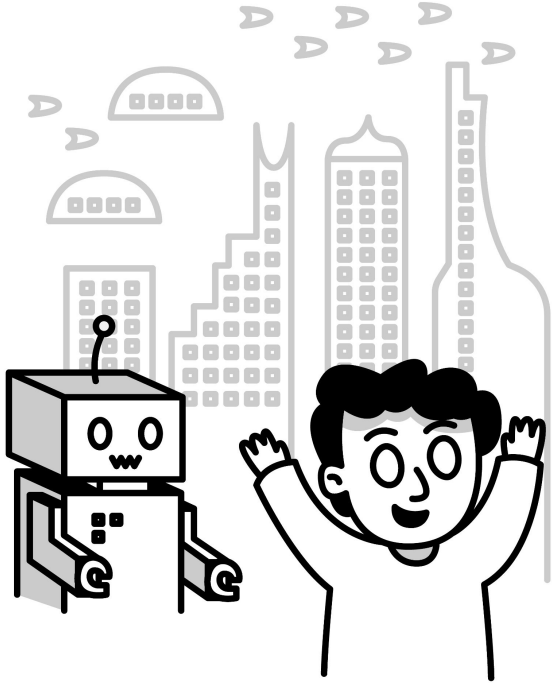
But where should a field go?

```
{
  "resourceLogs": [{
    "resource": {
      "attributes": [{
        "key": "service.name",
        "value": { "stringValue": "auth" }
      }]
    },
    "scopeLogs": [{
      "logRecords": [{
        "timeUnixNano": "1758326400000000000",
        "severityNumber": 9,
        "body": { "stringValue": "user logged in" }
      }]
    }]
  }]
}
```



So what are we actually doing here?

We’re paying all this cost, to think, to parse, but what for? The big promise: ***uniformity***



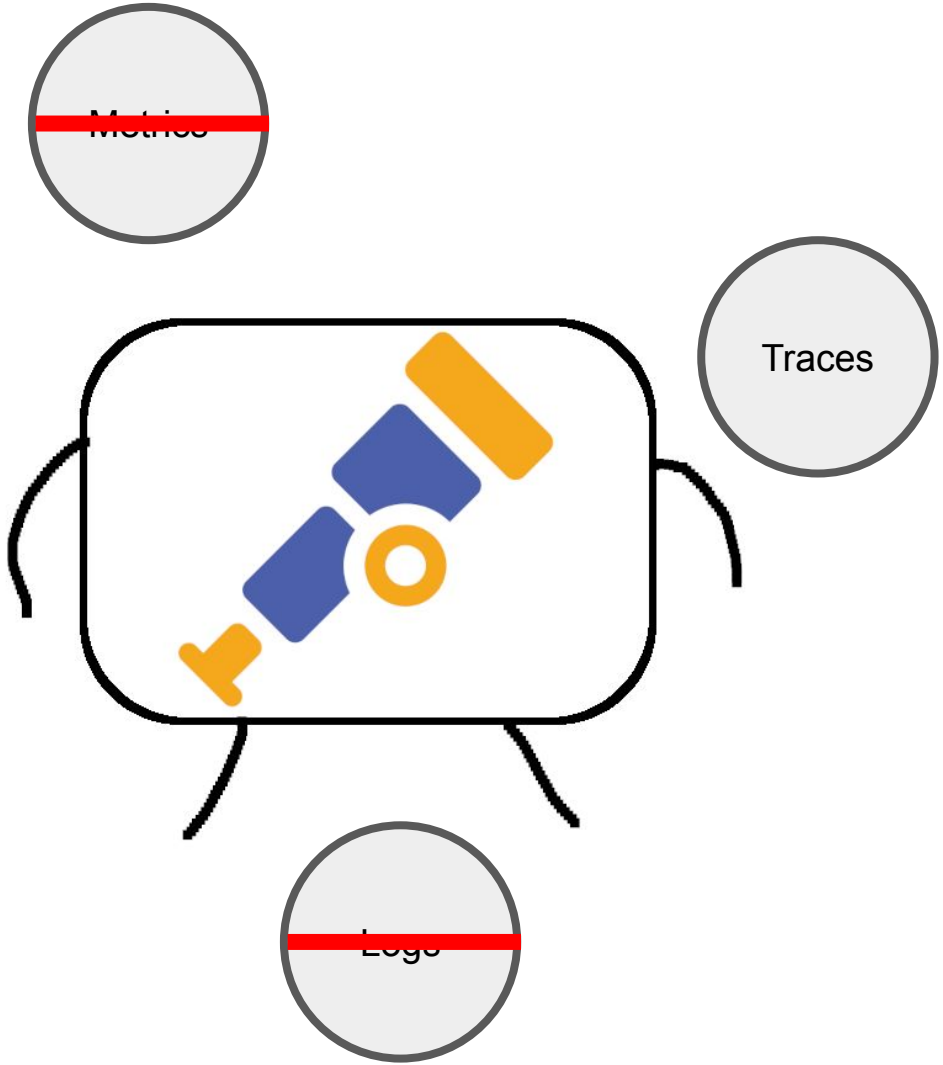
OTels lack of decisions makes for bad times

```
{
  "resourceLogs": [{
    "resource": {
      "attributes": [{
        "key": "service.name",
        "value": { "stringValue": "auth" }
      }]
    },
    "scopeLogs": [{
      "logRecords": [{
        "timeUnixNano": "1758326400000000000",
        "severityNumber": 9,
        "body": { "stringValue": "user logged in" }
      }]
    }]
  }]
}
```

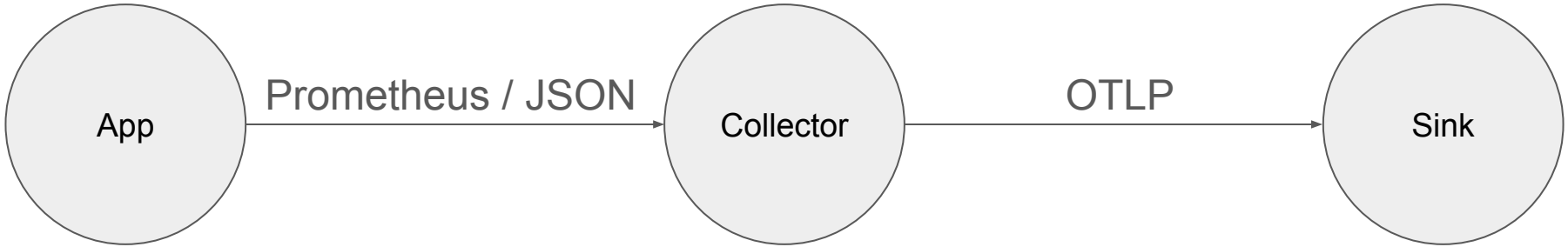

What's that thing?

By not being prescriptive, the spec leaves things open for interpretation, so uniformity is out




Traces

Maybe consider using a collector



Thanks!

Slides

Art by Alicia Edwards
https://www.aliciaillustration.com/

@colindou.ch on BlueSky